

# AI Assisted Coding Assignment- 10.2

RUDROJU RUPA SRI

2303A51918

BATCH-30

## **Lab 10** – Code Review and Quality: Using AI to Improve Code Quality and Readability

### Lab Objectives

- Use AI for automated code review and quality enhancement.
- Identify and fix syntax, logical, performance, and security issues in Python code.

- Improve readability and maintainability through structured

Week5 -

Tuesday

refactoring and comments.

- Apply prompt engineering for targeted improvements.
- Evaluate AI-generated suggestions against PEP 8 standards and software engineering best practices.

### **Task Description -1(Error Detection and Correction)**

#### **Task:**

Use AI to analyze a Python script and correct all syntax and logical errors.

Sample Input Code:

```
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total
```

Expected Output-1:

Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

The screenshot shows a VS Code editor with a file named 'AI ASS-10.2.py'. The code is a Python function `calculate_total(nums)` that sums a list of numbers. The original code has several errors: a missing colon at the end of the function definition, an undefined variable `sum` used for accumulation, and a missing colon at the end of the for loop. The AI prompt asks to analyze and correct these errors. The corrected code is shown below:

```

1 #Task-1(Error Detection and Correction)
2 #def calculate_total(nums)
3 # sum = 0
4 # for n in nums
5 # sum += n
6 # return total
7 #Prompt:Analyze a Python script and correct all syntax and logical errors and generate executable code.
8 # The original code has several syntax and logical errors:
9 # 1. The function definition is missing a colon (:) at the end of the line.
10 # 2. The code uses sum to accumulate the total, but the function returns an undefined variable.
11 # 3. The for loop is missing a colon (:) at the end of the line.
12 def calculate_total(nums):
13     # 2. The variable 'sum' is used to accumulate the total, but it is not returned at the end of the function.
14     total = 0 # Initialize total to 0
15     for n in nums: # Iterate through each number in the input list 'nums'
16         total += n # Add each number to the total
17     return total # Return the final total
18 # Test cases
19 print(calculate_total([1, 2, 3, 4])) # Expected: 10
20
21

```

The terminal at the bottom shows the command to run the script: `PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/AI ASS-10.2.py"`, which outputs `10`.

## Task Description -2(Code Style Standardization)

### Task:

Use AI to refactor Python code to comply with standard coding style guidelines.

Sample Input Code:

```
def findSum(a,b):return a+b
print(findSum(5,10))
```

Expected Output-2:

Well-structured, consistently formatted Python code following standard style conventions.

The screenshot shows a VS Code editor with a file named 'AI ASS-10.2.py'. The code is a Python function `findSum(a,b)` that returns the sum of two numbers. The original code is not formatted according to standard conventions. The AI prompt asks to refactor the code to comply with standard coding style guidelines. The refactored code is shown below:

```

20
21
22 #Task-2(Code Style Standardization)
23 def findSum(a,b):return a+b
24 #print(findSum(5,10))
25 #Prompt:refactor Python code to comply with standard coding style guidelines(No need of explanation)
26 def find_sum(a, b):
27     return a + b
28 print(find_sum(5, 10))
29
30

```

The terminal at the bottom shows the command to run the script: `PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/AI ASS-10.2.py"`, which outputs `15`.

## Task Description -3(Code Clarity Improvement)

### Task:

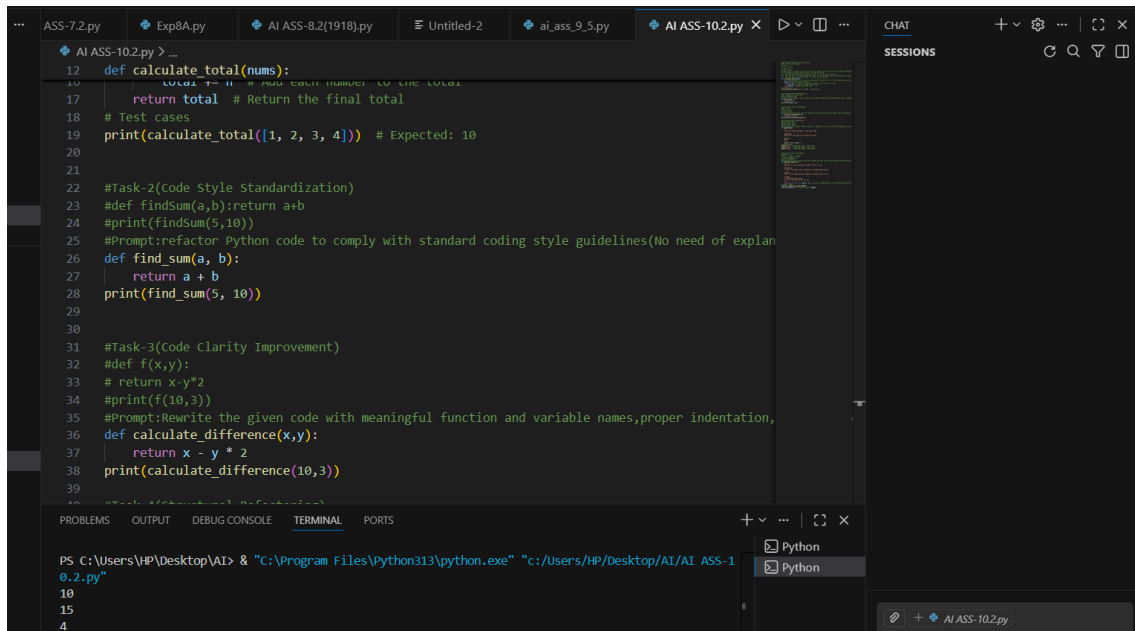
Use AI to improve code readability without changing its functionality.

Sample Input Code:

```
def f(x,y):  
    return x-y*2  
print(f(10,3))
```

Expected Output-3:

Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity.



```
12 def calculate_total(nums):  
13     total = 0 # Add each number to the total  
14     for num in nums:  
15         total += num  
16     return total # Return the final total  
17  
18 # Test cases  
19 print(calculate_total([1, 2, 3, 4])) # Expected: 10  
20  
21  
22 #Task-2(Code Style Standardization)  
23 def findSum(a,b):return a+b  
24 #print(findSum(5,10))  
25 #Prompt:refactor Python code to comply with standard coding style guidelines(No need of explan  
26 def find_sum(a, b):  
27     return a + b  
28 print(find_sum(5, 10))  
29  
30  
31 #Task-3(Code clarity Improvement)  
32 def f(x,y):  
33     # return x-y*2  
34     #print(f(10,3))  
35 #Prompt:Rewrite the given code with meaningful function and variable names,proper indentation,  
36 def calculate_difference(x,y):  
37     return x - y * 2  
38 print(calculate_difference(10,3))  
39
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/AI ASS-1  
0.2.py"  
10  
15  
4
```

#### Task Description -4(Structural Refactoring)

Task:

Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

```
print("Hello Ram")  
print("Hello Sita")  
print("Hello Ravi")
```

Expected Output-4:

Modular Python code using reusable functions to eliminate repetition.

```
39
40 #Task-4(Structural Refactoring)
41 #print("Hello Ram")
42 #print("Hello Sita")
43 #print("Hello Ravi")
44 #Prompt:Generate a Modular Python code using reusable functions to eliminate repetition with p
45 def greet(name):
46     """
47     Print a greeting message for the given name.
48
49     Parameters:
50     name (str): The name of the person to greet.
51
52     Returns:
53     None
54     """
55     print(f"Hello {name}")
56 # Example usage
57 greet("Ram") # Expected output: "Hello Ram"
58 greet("Sita") # Expected output: "Hello Sita"
59 greet("Ravi") # Expected output: "Hello Ravi"
60
61
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/ASS-10.2.py"

10  
15  
4  
Hello Ram  
Hello Sita  
Hello Ravi

Python  
Python

AI ASS-10.2.py  
Describe what to build next  
Auto

#### Task Description -5(Efficiency Enhancement)

Task:

Use AI to optimize Python code for better performance.

Sample Input Code:

```
numbers = []  
for i in range(1, 500000):  
    numbers.append(i * i)  
print(len(numbers))
```

Expected Output-5:

Optimized Python code that achieves the same result with improved performance.

```
62 #Task-5(Efficiency Enhancement)
63 #numbers = [ ]
64 #for i in range(1, 500000):
65 # numbers.append(i * i)
66 #print(len(numbers))
67 #Prompt:Optimize the Python code that achieves the same result with improved performance with
68 def generate_squares(n):
69     """
70     Generate a list of squares of numbers from 1 to n-1.
71
72     Parameters:
73     n (int): The upper limit (exclusive) for generating squares.
74
75     Returns:
76     list: A list containing the squares of numbers from 1 to n-1.
77
78     Example:
79     >>> generate_squares(10)
80     [1, 4, 9, 16, 25, 36, 49, 64, 81]
81     """
82     return [i * i for i in range(1, n)] # Using list comprehension for efficient generation o
83 # Example usage
84 squares = generate_squares(500000)
85 print(len(squares)) # Expected output: 499999
```

0.2.py~  
10  
15  
4  
Hello Ram  
Hello Sita  
Hello Ravi  
499999  
PS C:\Users\HP\Desktop\AI>

Ln 84, Col 35 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live